

Programare Orientată Pe Obiecte

Curs 7 - partea 2

Tipuri de clase

Clasele Object și Class

Exerciții

Noiembrie 2025

Tipuri de clase

Clasa de bază (Standard Class)

Definiție: O clasă standard care modelează obiecte cu proprietăți și comportamente.

```
public class Animal {  
    String name;  
    int age;  
  
    public Animal(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
  
    public void makeSound() {  
        System.out.println("Animal sound");  
    }  
}
```

Clase interne (Inner Classes)

Clasele interne sunt clase definite în interiorul altei clase. Ele sunt utilizate pentru:

- Organizarea codului.
- Accesul la membrii clasei exterioare.
- Encapsularea logicii relevante doar în contextul unei clase.

Există patru tipuri principale:

- 1 Clase interne non-stactice.
- 2 Clase interne statice.
- 3 Clase locale.
- 4 Clase anonime.

Clase interne non-stactice (Non-static Inner Class)

- Acestea au acces la membrii (inclusiv privați) ai clasei exterioare.
- Depind de o instanță a clasei exterioare.

Exemplu:

```
public class OuterClass {  
    private String message = "Hello";  
  
    public class InnerClass {  
        public void displayMessage() {  
            System.out.println(message);  
        }  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        OuterClass outer = new OuterClass();  
        OuterClass.InnerClass inner = outer.new InnerClass();  
        inner.displayMessage(); // Output: Hello  
    }  
}
```

Clase interne statice (Static Nested Class)

- Sunt statice, deci nu necesită o instanță a clasei exterioare.
- Pot accesa doar membrii statici ai clasei exterioare.

Exemplu:

```
public class OuterClass {  
    private static String staticMessage = "Hello";  
  
    public static class StaticNestedClass {  
        public void displayMessage() {  
            System.out.println(staticMessage);  
        }  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        OuterClass.StaticNestedClass nested = new OuterClass.  
            StaticNestedClass();  
        nested.displayMessage(); // Output: Hello  
    }  
}
```

Clase locale (Local Inner Class)

- Definite într-o metodă, constructor sau bloc.
- Au acces la variabilele finale sau efectiv finale ale metodei în care sunt definite.

Exemplu:

```
public class OuterClass {
    public void displayLocalClass() {
        class LocalInnerClass {
            public void print() {
                System.out.println("Local Inner Class");
            }
        }
        LocalInnerClass local = new LocalInnerClass();
        local.print();
    }
}

public class Main {
    public static void main(String[] args) {
        OuterClass outer = new OuterClass();
        outer.displayLocalClass(); // Output: Local Inner Class
    }
}
```

Clase anonime (Anonymous Inner Class)

- Clase fără nume, create pentru a implementa rapid interfețe sau clase abstracte.
- Folosite pentru suprascrierea metodelor pe loc.

Exemplu:

```
public class Main {  
    public static void main(String[] args) {  
        ArrayList<Shape> lst = new ArrayList<Shape>();  
        lst.add(new Circle(5, "Red"));  
        lst.add(new Circle(15, "Blue"));  
        lst.add(new Square("Blue", 4));  
        Collections.sort(lst, new Comparator<Shape>() {  
            @Override  
            public int compare(Shape s, Shape t) {  
                if (s.area() > t.area()) return 1;  
                if (s.area() < t.area()) return -1;  
                return 0;  
            }  
        });  
    }  
}
```

Alte tipuri de clasă

Clase abstracte Clase care nu pot fi instanțiate și care pot conține metode abstracte.

```
public abstract class Animal {  
    public abstract void makeSound();  
  
    public void sleep() {  
        System.out.println("Sleeping");  
    }  
}
```

Clasa finală O clasă care nu poate fi extinsă de alte clase.

```
public final class Utility {  
    // Codul clasei nu poate fi extins  
}
```

Alte tipuri de clasă

Clase imutabile (Immutable Class)

Definiție: Clase ale căror obiecte nu pot fi modificate după instanțiere.

```
public final class ImmutablePerson {  
    private final String name;  
    private final int age;  
  
    public ImmutablePerson(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public int getAge() {  
        return age;  
    }  
}
```

Alte tipuri de clasă

Clasă generică (Generic Classes); Clasă derivată (Subclass)

Clasă generică: Clasă care funcționează cu tipuri generice.

```
public class Box<T> {  
    private T item;  
  
    public void set(T item) {  
        this.item = item;  
    }  
  
    public T get() {  
        return item;  
    }  
}
```

Clasă derivată: Clasă care extinde o altă clasă (clasa părinte), moștenind membri și metode.

```
public class Dog extends Animal {  
    public void makeSound() {  
        System.out.println("Bark");  
    }  
}
```

Exemplu cu clase interne

Modelarea unei mașini folosind clase interne

- **Clasa Masina:** câmpuri (model, kilometraj), constructor, metodă `afiseazaRevizie()` cu clasă locală.
- **Clasa internă Motor:** `porneste()`, `opreste()`, `ruleaza(km)`.
- **Clasa statică MotorFactory:** creează diferite tipuri de specificații.
- **Clasa statică MotorSpecifications:** afișează caracteristicile motorului.
- **main:** creare Masina, Motor, apel metode + test MotorFactory.

Cerințe: Casa, Camera și Senzor

- **Clasa Casa:** câmp (adresa), constructor, metodă `afiseazaRaportCasa()` cu clasă locală.
- **Clasa internă Camera:** câmp (`numeCamera`), metodă `descriere()`.
- **Clasa internă Senzor** (în Camera): câmp (`tip`), metodă `citeste()`.
- **Clasa statică PlanCasa:** metodă `afiseazaPlanStandard()`.
- **Clasa statică CameraDefault:** câmpuri (`nume`, `suprafata`), metodă `info()`.
- **main:** creare Casa, Camera, Senzor + test PlanCasa.

Clasa Object

Clasa Object

Parinte pentru toate clasele din Java

Object definește versiuni implicite ale următoarelor metode:

- **toString()** – returnează un String (reprezentare "citibilă" a obiectului)
- **equals(Object obj)** – trebuie să fie suprascrisă pentru a realiza egalitatea de conținut în subclase
- **hashCode()** – returnează valoarea codului de dispersie pentru obiect; valorile sunt diferite pentru obiecte diferite
- **getClass()** – returnează un obiect de tipul Class; există un obiect de tipul Class pentru fiecare clasă dintr-o aplicație
- **notify(), notifyAll(), wait(), wait(long timeout), wait(long timeout, int nanos)** - folosite la multithreading
- **clone()** - creează și întoarce o copie a acestui obiect; "copie" poate depinde de clasa obiectului
- **finalize()** - destinat a efectua acțiuni de "curățare" înainte ca obiectul să fie irevocabil abandonat

Testarea Egalității

Există două feluri diferite de egalitate:

- 1 Egalitatea de identitate care înseamnă că două expresii au aceeași identitate (adică reprezintă același obiect)
 - Simbolul `==` testează egalitatea de identitate atunci când este aplicat datelor referință
- 2 Egalitatea de conținut care înseamnă că două expresii reprezintă obiecte cu aceeași valoare/conținut
 - Metoda `equals` din `Object` returnează `true` dacă și numai dacă este invocată cu două referințe identice; metoda poate fi suprascrisă în orice subclasă pentru a verifica egalitatea de conținut

Testarea Egalității

Exemplu

```
public class Circle extends Shape{
    int raza = 0;
    Circle(int raza, String culoarea){
        super(culoarea);
        this.raza = raza;
    }
    public double area(){ return Math.PI * raza * raza; }
    public boolean equals(Circle c){
        if (this.raza == c.raza) return true;
        return false;
    } }
public class Main {
    public static void main(String[] args) {
        Shape s1 = new Circle(5, "Red");
        Shape s2 = new Circle(5, "Red");
        if (s1.equals(s2)) System.out.print("Egalitate de continut");
        if (s1 == s2) System.out.print("Egalitate de referinta");
    } }
```

Particularități ale clasei Object

- 1 Este clasa Object o clasa abstracta ?
NU (are implementari de metode, cod pe care subclasele il pot folosi fara sa il suprascrie !)
- 2 Se pot suprascrie metodele din clasa Object ?
O parte din metode se pot suprascrie (ex: hashCode(), equals(), toString())
Alte metode (final) nu se pot suprascrie: getClass()
- 3 Daca e bine sa folosim polimorfismul, de ce nu facem ca toate metodele sa returneze Object ? Sau sa aiba parametrii Object ?
- type safety ex:

```
Object o = new Ferrari();  
o.goFast(); //Not legal!  
// putem chema doar metode definite in clasa Object !
```

Exemplu cu clasa Object

- Sistem de magazin online:
 - clase: `Produs`, `Client`, `Comanda`;
 - vrem un modul de **logare** care să poată primi *orice* tip de obiect.
- Fără `Object`, ar trebui:
 - metode separate: `logProdus(Produs p)`, `logClient(Client c)`, `logComanda(Comanda c)`, ...
- Cu `Object`:
 - o singură metodă: `logActiune(Object entitate)`.

A. Fără folosirea lui Object

```
class Produs {
    String nume;
    Produs(String nume) { this.nume = nume; }
}
class Client {
    String nume;
    Client(String nume) { this.nume = nume; }
}
// Logger cu metode separate
class LoggerA {
    void logProdus(Produs p) {
        System.out.println("[LOG PRODUS] " + p.nume);
    }
    void logClient(Client c) {
        System.out.println("[LOG CLIENT] " + c.nume);
    }
}
class ExempluA {
    public static void main(String[] args) {
        Produs p = new Produs("Laptop");
        Client c = new Client("Ana");
        LoggerA log = new LoggerA();
        log.logProdus(p);
        log.logClient(c);
    }
}
```

B. Cu Object (un singur logger generic)

```
class Produs {
    String nume;
    Produs(String nume) { this.nume = nume; }
    @Override public String toString() { return "Produs: " + nume; }
}

class Client {
    String nume;
    Client(String nume) { this.nume = nume; }
    @Override public String toString() { return "Client: " + nume; }
}

// Logger generic, primeste orice obiect
class LoggerB {
    void log(Object o) {
        System.out.println("[LOG] " + o);
    }
}

class ExempluB {
    public static void main(String[] args) {
        Produs p = new Produs("Laptop");
        Client c = new Client("Ana");
        LoggerB log = new LoggerB();
        log.log(p);    // foloseste toString() din Produs
        log.log(c);    // foloseste toString() din Client
    }
}
```

Comparație: fără Object vs. cu Object

- **Varianta A – fără Object**

- metode diferite pentru fiecare tip (`logProdus`, `logClient`, ...);
- dacă apare o nouă clasă, trebuie adăugată o nouă metodă.

- **Varianta B – cu Object**

- o singură metodă generică: `log(Object o)`;
- se bazează pe `toString()`, moștenită din `Object`;
- extensibilitate: orice nouă clasă poate fi logată imediat.

Clasa Class

Clasa Class

Clasa Class este definită astfel:

```
public final class Class extends Object
implements Serializable,
```

- Instanțele clasei Class reprezintă clase și interfețe dintr-o aplicație Java în curs de execuție
- Un obiect de tipul Class conține informații despre clasa a cărei instanță este obiectul care apelează
- Nu are constructor propriu
- Obiectele Class sunt construite la execuție de către JVM
- Există două moduri pentru a construi obiecte de acest tip:
 - getClass() din clasa Object
 - forName() din clasa Class (metodă statică)

Metoda getClass()

- Fiecare obiect moștenește aceeași metodă getClass() din clasa Object
- Această metodă este marcată final, deci nu poate fi suprascrisă
- O invocare a lui getClass() pe un obiect returnează o reprezentare numai pentru clasa care a fost folosită cu operatorul new pentru a crea obiectul
- Exemplu:

```
Shape p = new Circle(5, "Yellow");  
System.out.print(p.getClass()); // Circle
```

- Rezultatele a oricare două asemenea invocări pot fi comparate cu == sau != pentru a determina dacă ele reprezintă sau nu aceeași clasă:

```
(obiect1.getClass() == obiect2.getClass())
```

Operatorul **instanceof**

- Operatorul instanceof verifică dacă un obiect este de tipul dat ca al doilea argument al său
- Obiect instanceof NumeClasa: Va returna true dacă Obiect este de tipul NumeClasa; altfel va returna false
- Observați că aceasta înseamnă că va returna true dacă Obiect are tipul oricărei clase care este descendentă a lui NumeClasa
- Exemplu:

```
Shape c = new Circle(5, "yellow");  
System.out.print(c instanceof Circle); //true  
System.out.print(c instanceof Shape); //true
```

instanceof și getClass()

- Atât operatorul instanceof cât și metoda getClass() se pot folosi pentru a verifica clasa unui obiect
- Totuși, metoda getClass() este mai exactă
- Operatorul instanceof doar testează clasa unui obiect
- Metoda getClass() folosită într-un test cu == or != testează dacă două obiecte au fost instantiate din aceeași clasă

Exemplu instanceof și getClass()

Afișarea numelui unei clase folosind un obiect de tip Class:

```
void printClassName(Object obj) {  
    System.out.println(obj + " is of class " +  
        obj.getClass().getName());  
}
```

Alte exemple:

```
Circle c = new Circle(5, "red");  
printClassName(c); //Cercul cu raza 5 is of class Circle  
Class c1 = c.getClass();  
System.out.println(c1.getName()); // tipareste "Circle"  
Triangle t = new Triangle(7);  
printClassName(t);  
try {  
    Class c2 = Class.forName("Triangle");  
    System.out.println(c2.getName()); // tipareste "Triangle"  
} catch (ClassNotFoundException e) {  
    System.err.println("No class for \"Triangle\" " +  
        e.getMessage());  
}
```

Metode ale clasei **Class**

public String getName():

- returnează un String care reprezintă numele entității reprezentate de obiectul Class this
- entitatea poate fi: clasă, interfață, tablou, tip primitiv,

void public static Class.forName(String className) throws ClassNotFoundException:

- returnează un obiect de tipul Class care conține informații despre clasa obiectului

public Class[] getClasses():

- returnează un tablou de obiecte de tip Class;
- toate clasele și interfețele, membri publici ai clasei sunt reprezentate de acest obiect Class

Metode ale clasei **Class**

Field[] getFields:

- returnează un tablou care conține obiecte `Field` care reflectă toate câmpurile accesibile public ale clasei sau interfeței reprezentate de acest obiect `Class`

Method[] getMethods():

- returnează un tablou care conține obiecte `Method` care reflectă toate metodele publice membre ale clasei sau interfeței reprezentate de acest obiect `Class`, inclusiv cele declarate de clasă sau interfață și cele moștenite din superclase și superinterfețe

Constructor[] getConstructors():

- returnează un tablou care conține obiecte `Constructor` care reflectă toți constructorii publici ai clasei sau interfeței reprezentate de acest obiect `Class`

Exemplu sistem de plăți configurabil

Să se creeze o aplicație simplă de plăți. Tipul procesorului de plată (card, cash etc.) se decide dinamic, pe baza unui nume de clasă primit ca text.

- Să se definească o interfață `PaymentProcessor` cu metoda:
 - `void proceseaza(double suma);`
- Să se definească două implementări:
 - `CardPayment` – afișează că plata a fost făcută cu cardul;
 - `CashPayment` – afișează că plata a fost făcută cash.
- Să se definească o clasă `PaymentFactory` care:
 - primește un `String className`;
 - folosește `Class.forName(className)` pentru a încărca clasa;
 - creează dinamic o instanță cu reflecție;
 - verifică cu `instanceof` că obiectul este `PaymentProcessor`;
 - întoarce un `PaymentProcessor`.
- În `main`:
 - se dă numele complet al clasei (ex.: `"CardPayment"`);
 - se creează procesorul prin `PaymentFactory`;
 - se afișează tipul runtime folosind `obj.getClass()`;
 - se procesează o plată.

Interfața și implementările de bază

```
interface PaymentProcessor { void proceseaza(double suma); }

class CardPayment implements PaymentProcessor {
    @Override
    public void proceseaza(double suma) {
        System.out.println("Plata de " + suma + " RON cu CARD-ul.");
    }
    @Override
    public String toString() { return "CardPayment"; }
}

class CashPayment implements PaymentProcessor {
    @Override
    public void proceseaza(double suma) {
        System.out.println("Plata de " + suma + " RON CASH.");
    }
    @Override
    public String toString() { return "CashPayment"; }
}
```

Varianta A: fără Class.forName

```
class PaymentFactoryA {
    static PaymentProcessor create(String type) {
        if ("card".equalsIgnoreCase(type)) {
            return new CardPayment();
        }
        if ("cash".equalsIgnoreCase(type)) {
            return new CashPayment();
        }
        throw new IllegalArgumentException("Tip necunoscut: " + type);
    }
}

class ExempluA {
    public static void main(String[] args) {
        String tip = "card"; // sau "cash"
        PaymentProcessor p = PaymentFactoryA.create(tip);
        p.proceseaza(100.0);
        // pt un nou tip (ex. CryptoPayment), ce modificari se fac?
    }
}
```

Varianta B: cu `Class.forName` și `instanceof`

```
class PaymentFactoryB {
    static PaymentProcessor create(String className) throws Exception {
        // Incarcam clasa dupa numele ei (runtime)
        Class<?> clazz = Class.forName(className);
        // Cream instanta folosind constructorul fara parametri
        Object obj = clazz.newInstance();
        // Verificam tipul real al obiectului
        if (!(obj instanceof PaymentProcessor)) {
            throw new IllegalArgumentException("Clasa " + className + " nu
                este PaymentProcessor"); }
        return (PaymentProcessor) obj;
    }
}

class ExempluB {
    public static void main(String[] args) throws Exception {
        String numeClasa = "CardPayment";
        // sau: String numeClasa = "CashPayment";
        PaymentProcessor p = PaymentFactoryB.create(numeClasa);
        // Informatii despre tipul real (folosim clasa Class)
        System.out.println("Procesor creat:" + p.getClass().getSimpleName());
        p.proceseaza(150);
    }
}
```

Observații:

De ce folosim `Class<?>` ?

- În Java, `Class` este generic: `Class<T>`.
- La `Class.forName(String)`, tipul concret nu este cunoscut la compilare.
- În `Class<?>`: “?” înseamnă: „o clasă a unui tip necunoscut, dar valid”.
- Evităm folosirea tipului brut (raw type) `Class`, care produce avertismente.
- Nu putem folosi `Class<Object>` deoarece clasa încărcată poate fi `CardPayment`, `CashPayment` etc., nu neapărat `Object`.
- **Scop:** cod sigur, generic, fără presupuneri despre tipul real al clasei încărcate.

De ce folosim `Object obj = clazz.newInstance();`

- `clazz.newInstance()` întoarce întotdeauna un `Object`.
- Nu știm la compilare ce clasă va fi încărcată prin `Class.forName()`.
- `Object` este singurul tip comun garantat pentru toate clasele posibile.
- Apoi verificăm tipul real cu `instanceof` pentru siguranță.
- După verificare, putem face cast la `PaymentProcessor`.